

EduTrack System Documentation — Part 2

Backend to Frontend API, Validation & Communication Guide

1. Executive API Interface Overview

The interface between the Python FastAPI backend and the React Vite frontend operates entirely over asynchronous **RESTful HTTP/HTTPS APIs**. Data is exchanged strictly in standard **JSON** (JavaScript Object Notation) format. FastAPI handles route dispatching, CORS headers, and input/output payload validation through **Pydantic Schemas**.

Key Architectural Concept: FastAPI automatically validates incoming JSON request bodies against Pydantic schemas. If a client sends invalid data (e.g., missing required field or wrong datatype), FastAPI intercepts the call before it reaches business logic and returns a standardized HTTP 422 Unprocessable Entity response.

2. FastAPI Routers & Unified Exception Handling

API endpoints are grouped into dedicated router files located in `backend/app/routers/`. All routers are mounted in `main.py` under the `/api` prefix:

Router File	Prefix Path	HTTP Methods	Description & Responsibility
<code>routers/auth.py</code>	<code>/api/auth</code>	POST	User Registration, Login, and JWT Token issuance
<code>routers/states.py</code>	<code>/api/states</code>	GET, POST, PUT, DELETE	CRUD management for geographic States
<code>routers/districts.py</code>	<code>/api/districts</code>	GET, POST, PUT, DELETE	Districts management filtered by <code>state_id</code>
<code>routers/institutions.py</code>	<code>/api/institutions</code>	GET, POST, PUT, DELETE	Educational Institutions CRUD and type filter
<code>routers/courses.py</code>	<code>/api/courses</code>	GET, POST, PUT, DELETE	Academic Courses management per Institution
<code>routers/branches.py</code>	<code>/api/branches</code>	GET, POST, PUT, DELETE	Departmental Branches under Courses
<code>routers/academic_years.py</code>	<code>/api/academic-years</code>	GET, POST, PUT, DELETE	Academic Years & Strength tracking
<code>routers/syllabus.py</code>	<code>/api/syllabus</code>	GET, POST, DELETE	Syllabus entry text & PDF file upload handling
<code>routers/graph_data.py</code>	<code>/api/graph-data</code>	GET	Aggregated hierarchy graph (nodes & edges) for UI visualizer
<code>routers/bulk.py</code>	<code>/api/bulk</code>	POST	Bulk data import pipeline for institutions & hierarchy

3. Request & Response Validation via Pydantic Schemas

Pydantic schemas (in `backend/app/schemas/`) define contract models for input validation and output serialization. By setting `from_attributes = True` (formerly `orm_mode`), Pydantic seamlessly reads values directly from SQLAlchemy ORM models.

```
# Example schema hierarchy from backend/app/schemas/institution.py
from pydantic import BaseModel
from datetime import datetime

# 1. Base Schema (Shared attributes)
class InstitutionBase(BaseModel):
    name: str
    address: str | None = None
    district_id: int
    institution_type_id: int

# 2. Request Schema for Client Creation Payload
class InstitutionCreate(InstitutionBase):
    pass

# 3. Response Schema sent back to Frontend
class InstitutionResponse(InstitutionBase):
    id: int
    created_at: datetime
    class Config:
        from_attributes = True # Converts SQLAlchemy ORM object -> JSON dictionary
```

4. Security & JWT Authentication Pipeline

EduTrack enforces secure role-based authentication using **JSON Web Tokens (JWT)**:

1. **Login Request:** User submits credentials to POST /api/auth/token.
2. **Verification:** crud/user_crud.py fetches user and verifies password hash using passlib (bcrypt).
3. **JWT Issuance:** Server generates signed token containing user_id and expiration time.
4. **Header Injection:** Frontend stores token in localStorage and attaches Authorization: Bearer <token> to every Axios request.
5. **Endpoint Protection:** Protected routers require current_user: User = Depends(get_current_user).

5. Frontend Axios Client & Service Bindings

The React client configures a centralized Axios HTTP instance in frontend/src/api/client.js. Request interceptors automatically attach authentication headers, and domain-specific API wrappers organize remote procedure calls:

```
// frontend/src/api/client.js
import axios from 'axios';

const api = axios.create({ baseURL: '/api', headers: { 'Content-Type': 'application/json' } });

// Request Interceptor for Bearer Token
api.interceptors.request.use((config) => {
    const token = localStorage.getItem('edutrack_token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
export default api;
```

6. React UI Page Component Binding Matrix

Mapping between React Pages/Components and Backend API Endpoints:

React Page / Component	Frontend API Module	Backend Router Endpoint	UI Functionality
HomePage.jsx	hierarchyApi.getStates()	GET /api/states	State dropdown selector & high-level stats
HomePage.jsx	institutionApi.getInstitutions()	GET /api/institutions	Hierarchical tree drilldown of institutions
AnalyticsPage.jsx	graphApi.getGraphData()	GET /api/graph-data	Interactive network graph of hierarchy nodes/edges
SearchResultsPage.jsx	syllabusApi.searchSyllabus()	GET /api/syllabus/search	Search syllabus content & view PDF downloads
AdminConsolePage.jsx	hierarchyApi.createState()	POST /api/states	Admin management dashboard for full hierarchy CRUD
AuthModal.jsx	authApi.login()	POST /api/auth/token	User login dialog & JWT token store in localStorage